Documentación PyQt6 · IIC2233

Índice

1. Imports
2. Esqueleto de una aplicación
3. `QWidget`: ventanas y widgets
4. `QLabel`
5. `QLineEdit`
6. `QPushButton`
7. Imágenes: `QPixmap`
8. Layouts
9. Señales y *slots*
10. Eventos de mouse y teclado
11. Separación *front-end / back-end*
12. Varias ventanas
13. PyQt5 vs PyQt6
14. Otros widgets útiles
15. Buscar lo que no está acá

---

1. Imports

PyQt6 está dividido en módulos. Los tres que se usan siempre:

Módulo	Qué trae	Clases que vas a usar
`PyQt6.QtWidgets`	Los elementos gráficos	`QApplication`, `QWidget`, `QLabel`, `QLineEdit`, `QPushButton`, `QHBoxLayout`, `QVBoxLayout`, `QGridLayout`, `QMessageBox`
`PyQt6.QtCore`	Lo que no es gráfico: señales, constantes	`QObject`, `pyqtSignal`, `Qt`
`PyQt6.QtGui`	Imágenes y eventos	`QPixmap`, `QMouseEvent`, `QKeyEvent`

```
from PyQt6.QtWidgets import (
    QApplication, QWidget, QLabel, QLineEdit, QPushButton,
    QHBoxLayout, QVBoxLayout, QGridLayout,
)
from PyQt6.QtCore import QObject, pyqtSignal, Qt
from PyQt6.QtGui import QPixmap
```

> `Qt` (con las constantes `Qt.AlignmentFlag`, `Qt.AspectRatioMode`, `Qt.Key`, `Qt.MouseButton`) se importa desde **`QtCore`**, no desde `QtWidgets`.

---

2. Esqueleto de una aplicación

```
import sys
from PyQt6.QtWidgets import QApplication, QWidget


class MiVentana(QWidget):
    def __init__(self) -> None:
        super().__init__()
        self.setGeometry(200, 100, 300, 300)   # x, y, ancho, alto
        self.setWindowTitle('Mi ventana')


if __name__ == '__main__':
    def hook(type_, value, traceback) -> None:
        # Imprimimos el error con el hook original de Python. Al instalar
        # nuestro propio sys.excepthook, PyQt no cierra el programa
        sys.__excepthook__(type_, value, traceback)

    sys.excepthook = hook

    app = QApplication([])
    ventana = MiVentana()
    ventana.show()
    sys.exit(app.exec())
```

`QApplication`

Uso	Qué hace
`app = QApplication([])`	Crea la aplicación. **Va antes que cualquier widget** y existe **una sola** por programa, tenga las ventanas que tenga.
`app.exec()`	Inicia el *event loop*: desde aquí el programa queda esperando eventos (clics, teclas…). **Bloquea** hasta que se cierra la aplicación y retorna un código de salida (`0` = sin errores).
`sys.exit(app.exec())`	Forma habitual de terminar: sale del programa con el código que retornó la app.
`QApplication.quit`	Cierra la aplicación. Se puede conectar a un botón: `boton.clicked.connect(QApplication.quit)` .

Todo lo que configures (widgets, conexiones de señales, `show()` de la primera ventana) tiene que estar listo **antes** de `app.exec()` .

`sys.excepthook`

Sin el *hook*, cuando ocurre un error dentro de un *slot* PyQt puede **cerrar el programa sin mostrar nada en consola**. Con el *hook* del esqueleto el error se imprime con su *traceback* normal y la ventana sigue abierta. Va **antes** de crear la `QApplication` .

Sistema de coordenadas

El origen `(0, 0)` está en la **esquina superior izquierda**; `x` crece hacia la derecha e `y` **hacia abajo**. Para una ventana el origen es la esquina de la pantalla; para un widget dentro de una ventana, la esquina de su widget padre.

---

3. `QWidget`: ventanas y widgets

`QWidget` es la clase base de **todos** los elementos gráficos. Una ventana es un `QWidget` sin padre; un botón o un label son `QWidget` con padre. Todo lo de esta tabla lo heredan `QLabel`, `QLineEdit`, `QPushButton`, etc.

Método	Qué hace	Ejemplo
`QWidget(padre=None)`	Constructor. Si recibe un padre, el widget se dibuja **dentro** de él. Sin padre (y sin layout) no aparece.	`QLabel('Hola', self)`
`setGeometry(x, y, ancho, alto)`	Posición y tamaño.	`self.setGeometry(300, 300, 300, 200)`
`move(x, y)`	Solo la posición.	`self.label.move(10, 15)`
`resize(ancho, alto)`	Solo el tamaño.	`boton.resize(boton.sizeHint())`
`sizeHint()`	Tamaño sugerido por Qt para el widget según su contenido.	
`x()`, `y()`, `width()`, `height()`	Posición y tamaño actuales, en píxeles.	`ancho = self.width()`
`geometry()`	El rectángulo del widget (`QRect`), relativo a su padre. Con `.contains(punto)` dice si un punto cae dentro.	`self.label.geometry().contains(p)`
`setFixedSize(ancho, alto)`	Tamaño fijo: el usuario no puede redimensionar.	
`setMinimumSize(w, h)` / `setMaximumSize(w, h)`	Límites de tamaño (también `setMaximumWidth`, `setMaximumHeight`).	
`setWindowTitle(texto)`	Título de la ventana.	`self.setWindowTitle('Gestor')`
`show()`	Muestra el widget o la ventana.	`ventana.show()`
`hide()`	Lo oculta **sin destruirlo**; se puede volver a mostrar con `show()`.	`self.hide()`
`close()`	Cierra la ventana. Si era la última visible, la aplicación termina.	
`isVisible()`	`True` si está visible.	
`setEnabled(bool)`	Habilita o deshabilita. Un botón deshabilitado se ve gris y **no emite** `clicked`.	`boton.setEnabled(False)`

Método	Qué hace	Ejemplo
`isEnabled()`	`True` si está habilitado.	
`setLayout(layout)`	Define el layout principal del widget (§8).	`self.setLayout(grilla)`
`setStyleSheet(css)`	Estilo con sintaxis tipo CSS.	`label.setStyleSheet('background-color: white; border: 1px solid black;')`
`sender()`	Dentro de un *slot*: el objeto que emitió la señal (§9.3).	`boton = self.sender()`
`underMouse()`	`True` si el cursor está sobre el widget.	`if self.label.underMouse(): ...`
`childAt(punto)`	El widget hijo que está en esa posición (o `None`).	`self.childAt(event.position().toPoint())`
`setMouseTracking(bool)`	Si es `True`, `mouseMoveEvent` se llama aunque no haya botón presionado.	
`setFocus()`	Le da el foco del teclado al widget.	`campo.setFocus()`

`setStyleSheet` recibe un `str` con propiedades `nombre: valor;`. Las más usadas:

Propiedad	Ejemplo
`background-color`	`background-color: white;` · `background-color: #ffeeaa;`
`color` (del texto)	`color: red;`
`border`	`border: 1px solid black;` (grosor, estilo, color)
`border-radius`	`border-radius: 6px;` (esquinas redondeadas)
`font-size` / `font-weight`	`font-size: 18px;` · `font-weight: bold;`
`padding`	`padding: 4px;` (espacio entre el borde y el contenido)

Se pueden combinar varias en el mismo `str`: `label.setStyleSheet('background-color: white; border: 1px solid black; font-size: 18px;')`. Los colores aceptan nombres en inglés (`white`, `lightblue`…) o hexadecimal (`#rrggbb`).

> **Guarda como atributo (`self.algo`) todo widget que vayas a usar después** de crearlo. Una variable local se pierde al terminar el método y ya no puedes leerla ni cambiarla.

---

4. `QLabel`

Muestra **texto no editable** o una **imagen**. **No tiene señal `clicked`**: para reaccionar a un clic sobre un label se usa `mousePressEvent` de la ventana (§10.1).

Método	Qué hace
`QLabel(texto, padre)`	Crea el label con un texto inicial. `QLabel(padre)` lo crea vacío.

Método	Qué hace
`setText(texto)`	Cambia el texto. Recibe un `str`: para un número, `setText(str(n))`.
`text()`	Retorna el texto actual.
`clear()`	Borra el contenido (texto o imagen).
`setPixmap(pixmap)`	Muestra una imagen en vez de texto (§7).
`setScaledContents(True)`	Estira la imagen para que llene el tamaño del label.
`setAlignment(alineacion)`	Alineación del contenido.
`setWordWrap(True)`	Corta el texto en varias líneas si no cabe.

Alineaciones (`from PyQt6.QtCore import Qt`):

Constante	Alinea
`Qt.AlignmentFlag.AlignCenter`	Centro (horizontal y vertical)
`Qt.AlignmentFlag.AlignLeft` / `AlignRight` / `AlignHCenter`	Horizontal
`Qt.AlignmentFlag.AlignTop` / `AlignBottom` / `AlignVCenter`	Vertical

Se combinan con `|`: `Qt.AlignmentFlag.AlignLeft | Qt.AlignmentFlag.AlignTop`.

```
self.label_estado = QLabel('', self)
self.label_estado.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.label_estado.setText(f'Puntaje: {puntaje}')
```

> Si en algún sistema operativo un `setText` no se refleja de inmediato, `label.repaint()` fuerza el redibujo.

---

5. `QLineEdit`

Cuadro de texto de **una línea** para que el usuario escriba.

Método / señal	Qué hace
`QLineEdit(texto_inicial, padre)`	Crea el campo. `QLineEdit('', self)` lo deja vacío.
`text()`	Retorna lo escrito, **siempre como** `str` (vacío: `''`). Para un número: `int(campo.text())`, que puede lanzar `ValueError`.
`setText(texto)`	Escribe en el campo.
`clear()`	Lo vacía.
`setPlaceholderText(texto)`	Texto gris de ayuda que se ve solo cuando el campo está vacío y **no** es parte de `text()`.
`setEchoMode(QLineEdit.EchoMode.Password)`	Muestra puntos en vez de caracteres (contraseñas).
`setMaxLength(n)`	Máximo de caracteres.
`setReadOnly(True)`	No editable.
Señal `textChanged(str)`	Se emite cada vez que cambia el texto, con el texto nuevo.

Método / señal	Qué hace
Señal `returnPressed()`	Se emite al apretar Enter dentro del campo.
Señal `editingFinished()`	Se emite al apretar Enter o al salir del campo.

```
self.campo_nombre = QLineEdit('', self)
self.campo_nombre.setPlaceholderText('Nombre')
...
nombre = self.campo_nombre.text().strip()
if not nombre:      # '' es falsy
    ...
```

---

6. `QPushButton`

Método / señal	Qué hace
`QPushButton(texto, padre)`	Crea el botón.
Señal `clicked`	Se emite al hacer clic (presionar y soltar sobre el botón). Se conecta con `boton.clicked.connect(self.metodo)`. Envía un `bool` (`checked`, casi siempre `False`) que puedes ignorar (§9.4).
Señales `pressed` / `released`	Se emiten al presionar / al soltar el botón, por separado.
`setText(texto)` / `text()`	Cambia / lee el texto.
`setEnabled(False)`	Lo deshabilita: no se puede apretar y no emite `clicked`.
`resize(boton.sizeHint())`	Tamaño justo para su texto (cuando no usas layout).
`setFixedSize(w, h)`	Tamaño fijo.
`setIcon(QIcon(ruta))`	Ícono (`QIcon` está en `QtGui`).

El carácter `&`. Un `&` en el texto **no se muestra**: marca la letra siguiente como atajo de teclado (`'&Jugar'` se ve como *Jugar* y se activa con **Alt+J**). Para que aparezca un `&` de verdad se escribe `&&`. Ojo: `text()` retorna el texto **con** el `&` (`'&Jugar'`).

```
self.boton_enviar = QPushButton('&Enviar', self)
self.boton_enviar.clicked.connect(self.enviar)    # sin paréntesis
```

---

7. Imágenes: `QPixmap`

Una imagen se carga en un `QPixmap` (de `QtGui`) y **se muestra dentro de un `QLabel`**.

Método	Qué hace
`QPixmap(ruta)`	Carga la imagen desde un archivo (`.png`, `.jpg`...).
`isNull()`	`True` si **no se pudo cargar** (ruta mala). PyQt **no lanza error**: el label simplemente queda vacío.

Método	Qué hace
`scaled(ancho, alto, modo)`	Retorna **una copia** escalada (el original no cambia).
`width()` / `height()`	Dimensiones en píxeles.
`label.setPixmap(pixmap)`	Pone la imagen en el label.
`label.setScaledContents(True)`	Alternativa a `scaled`: estira la imagen al tamaño del label (puede deformarla).

Modos de `scaled` (`Qt.AspectRatioMode`):

Constante	Resultado
`Qt.AspectRatioMode.KeepAspectRatio`	Cabe en el rectángulo **sin deformarse**
`Qt.AspectRatioMode.IgnoreAspectRatio`	Llena el rectángulo exacto, deformándose si hace falta
`Qt.AspectRatioMode.KeepAspectRatioByExpanding`	Cubre el rectángulo sin deformarse; puede sobrar imagen

Opcionalmente, un cuarto argumento `Qt.TransformationMode.SmoothTransformation` mejora la calidad.

```
import os
from PyQt6.QtGui import QPixmap
from PyQt6.QtCore import Qt

ruta = os.path.join('assets', 'logo.png')
pixeles = QPixmap(ruta)
self.label_imagen = QLabel(self)
self.label_imagen.setGeometry(10, 10, 80, 80)
self.label_imagen.setPixmap(pixeles.scaled(80, 80, Qt.AspectRatioMode.KeepAspectRatio))
```

Rutas. Usa siempre `os.path.join('carpeta', 'archivo.png')` en vez de escribir `/` o `\`. La ruta es **relativa a la carpeta desde la que ejecutas** `python`, no a la del archivo `.py`: corre el programa con la terminal parada en la carpeta que contiene `main.py`.

> Puedes cargar cada imagen **una sola vez** (por ejemplo en un diccionario `{'nombre': QPixmap(...)}` creado en `__init__`) y reutilizarla en varios labels.

---

8. Layouts

Un layout **ubica y redimensiona automáticamente** los widgets cuando cambia el tamaño de la ventana. La alternativa es la posición absoluta con `setGeometry` / `move`, que no se adapta.

Las dos formas se pueden usar, pero no conviene mezclarlas en el mismo widget: si una ventana tiene `setLayout(...)`, el layout pisa las posiciones que hayas dado con `setGeometry` a los widgets que agregues a él.

8.1 `QHBoxLayout` y `QVBoxLayout`

Ordenan los widgets en una **fila** (H) o en una **columna** (V).

Método	Qué hace
`addWidget(widget)`	Agrega un widget al final.
`addLayout(layout)`	Mete un layout dentro de otro (así se combinan filas y columnas).
`addStretch(n)`	Agrega espacio flexible que empuja a los widgets. Varios `addStretch` se reparten el espacio según su `n`.
`addSpacing(px)`	Espacio fijo.
`setSpacing(px)`	Separación entre elementos.
`setContentsMargins(izq, arriba, der, abajo)`	Márgenes del layout.

```
fila = QHBoxLayout()
fila.addStretch(1)
fila.addWidget(self.boton_aceptar)
fila.addWidget(self.boton_cancelar)

columna = QVBoxLayout()
columna.addWidget(self.label_titulo)
columna.addStretch(1)
columna.addLayout(fila)

self.setLayout(columna)
```

8.2 `QGridLayout`

Grilla de **filas y columnas**, empezando en `(0, 0)`.

Método	Qué hace
`addWidget(widget, fila, columna)`	Pone el widget en esa celda.
`addWidget(widget, fila, columna, n_filas, n_columnas)`	Lo extiende por varias celdas. `addWidget(label, 3, 0, 1, 3)` ocupa la fila 3 entera de una grilla de 3 columnas.
`addLayout(layout, fila, columna)`	Pone un layout en una celda.
`indexOf(widget)`	Índice del widget dentro del layout: **el orden en que se agregó** (0, 1, 2…). `-1` si no está.
`getItemPosition(indice)`	Retorna la tupla `(fila, columna, n_filas, n_columnas)` del elemento con ese índice.
`itemAtPosition(fila, columna)`	El elemento de esa celda; con `.widget()` se obtiene el widget.
`count()`	Cantidad de elementos.
`setSpacing(px)`	Separación entre celdas.

Recorrer una lista plana como grilla de `n` columnas: `fila = i // n`, `columna = i % n`.

```
self.grilla = QGridLayout()
valores = ['1', '2', '3', '4', '5', '6', '7', '8', '9', '*', '0', '#']
for i, valor in enumerate(valores):
    boton = QPushButton(valor, self)
    boton.clicked.connect(self.boton_clickeado)
    self.grilla.addWidget(boton, i // 3, i % 3)
self.setLayout(self.grilla)

def boton_clickeado(self) -> None:
    boton = self.sender()
    indice = self.grilla.indexOf(boton)                    # 0..11, orden de inserción
    fila, columna, _, _ = self.grilla.getItemPosition(indice)
```

> Si después de la grilla agregas otros widgets al mismo layout (un label de estado, por ejemplo), también reciben un índice: `indexOf` cuenta **todo** lo agregado a ese layout, en orden.

---

9. Señales y *slots*

Una **señal** avisa que algo pasó; un **slot** es la función o método que se ejecuta cuando la señal se emite. Se unen con `connect`.

9.1 Conectar

```
boton.clicked.connect(self.procesar)       # ✓ entrega la función
boton.clicked.connect(self.procesar())     # ✗ la ejecuta ahora y conecta su resultado (None)
```

Método	Qué hace
`senal.connect(slot)`	Cada vez que la señal se emita, se llama a `slot`. Una señal puede tener **varios** slots conectados y un slot puede recibir **varias** señales.
`senal.disconnect()`	Desconecta todos los slots de esa señal (o `disconnect(slot)`, solo ese).

Un slot puede ser cualquier cosa llamable: un método propio, una función, una `lambda`, o un método de PyQt como `ventana.show`, `ventana.hide`, `ventana.close`, `label.setText` o `otra_senal.emit`.

9.2 Señales propias: `pyqtSignal`

```
from PyQt6.QtCore import QObject, pyqtSignal


class Logica(QObject):
    senal_resultado = pyqtSignal(str)          # atributo de CLASE
    senal_posicion = pyqtSignal(int, int)

    def __init__(self) -> None:
        super().__init__()                     # obligatorio

    def procesar(self, x: int, y: int) -> None:
        self.senal_posicion.emit(x, y)         # emite con datos
        self.senal_resultado.emit('listo')
```

Reglas que **sí o sí** hay que cumplir:

Regla	Si no se cumple
La señal se declara como **atributo de clase**, no dentro de `__init__`.	`AttributeError: 'PyQt6.QtCore.pyqtSignal' object has no attribute 'connect'`
La clase hereda de **`QObject`** (o de un widget, que ya hereda de `QObject`). Un *back-end* sin parte gráfica hereda de `QObject`.	La señal no se puede conectar ni emitir.
El `__init__` llama a **`super().__init__()`**.	`RuntimeError: super-class __init__() of type ... was never called`
`emit` recibe **la misma cantidad y tipo** de argumentos que se declararon.	`TypeError: ... signal has 2 argument(s) but 1 provided` o `... has unexpected type`
El nombre de la señal usa solo caracteres ASCII (`senal`, no `señal`).	`SystemError` al definir la clase.

Tipos que se pueden declarar: `int`, `float`, `str`, `bool`, `list`, `dict`, `tuple`, `object` (cualquier cosa) y también clases propias. `pyqtSignal()` sin argumentos es una señal que no manda datos.

El *slot* recibe los datos como parámetros, en el mismo orden:

```
logica.senal_posicion.connect(ventana.mostrar_posicion)

# en la ventana:
def mostrar_posicion(self, x: int, y: int) -> None:
    self.label.setText(f'{x}, {y}')
```

Si el slot tiene **menos** parámetros que la señal, PyQt le pasa solo los primeros y descarta el resto sin avisar; si tiene **más**, falla con `TypeError` al emitir.

9.3 Saber quién emitió la señal: `sender()`

Cuando **varios widgets comparten el mismo slot**, dentro del slot `self.sender()` retorna el objeto que emitió la señal. Con él puedes leer su texto, buscar su posición en un layout (`self.grilla.indexOf(boton)`) o buscarlo en

una lista (`self.botones.index(boton)`).

```
def boton_clickeado(self) -> None:
    boton = self.sender()
    texto = boton.text()
```

`sender()` solo tiene sentido **dentro de un slot** que fue llamado por una señal; llamado directamente retorna `None` .

9.4 Pasar datos extra a un slot: `lambda`

Otra forma de saber qué botón se apretó es "fijar" el dato al conectar:

```
for i, boton in enumerate(self.botones):
    boton.clicked.connect(lambda _, indice=i: self.senal_elegir.emit(indice))
```

- `indice=i` congela el valor de `i` **de esa vuelta**. Sin eso, todas las lambdas usan el último valor de `i` .
- El `_` recibe el `bool` que manda `clicked` . **Sin el `_` , ese `False` se mete en `indice` .**

9.5 Conectar una señal directo a otra

Un widget puede reenviar un evento como señal propia sin escribir un método intermedio:

```
self.boton.clicked.connect(self.senal_boton.emit)    # senal_boton = pyqtSignal()
```

Funciona si la señal propia no declara argumentos: PyQt descarta el `bool` de `clicked` .

---

10. Eventos de mouse y teclado

Todo `QWidget` tiene métodos que Qt llama solo cuando ocurre un evento. Para reaccionar se **sobrescriben** en tu clase; no se conectan ni se llaman a mano. El nombre tiene que ser **exacto** (`mousePressEvent` , con mayúsculas): si te equivocas, nunca se ejecuta y no hay error. Cada uno recibe un objeto `event` con la información de lo que pasó.

Todos los que sirven en el curso:

Método a sobrescribir	**Cuándo se llama**	**Tipo del `event` (`QtGui`)**
`mousePressEvent(self, event)`	Al presionar un botón del mouse sobre el widget	`QMouseEvent`
`mouseReleaseEvent(self, event)`	Al soltarlo	`QMouseEvent`
`mouseDoubleClickEvent(self, event)`	Doble clic	`QMouseEvent`
`mouseMoveEvent(self, event)`	Al mover el mouse con un botón presionado (o siempre, con `setMouseTracking(True)`)	`QMouseEvent`
`wheelEvent(self, event)`	Al girar la rueda del mouse	`QWheelEvent`
`enterEvent(self, event)` / `leaveEvent(self, event)`	Cuando el mouse **entra** al widget / **sale** de él	`QEnterEvent` / `QEvent`

Método a sobrescribir	Cuándo se llama	Tipo del `event` (`QtGui`)
`keyPressEvent(self, event)`	Al presionar una tecla	`QKeyEvent`
`keyReleaseEvent(self, event)`	Al soltarla	`QKeyEvent`
`closeEvent(self, event)`	Cuando se intenta cerrar la ventana (con la X o con `close()`)	`QCloseEvent`
`showEvent(self, event)` / `hideEvent(self, event)`	Al mostrarse / ocultarse	`QShowEvent` / `QHideEvent`
`resizeEvent(self, event)`	Al cambiar de tamaño	`QResizeEvent`

Los tipos del `event` solo hacen falta para las anotaciones (`event: QMouseEvent`); se importan desde `PyQt6.QtGui`, salvo `QEvent`, que está en `PyQt6.QtCore`.

En `closeEvent`, `event.accept()` deja que la ventana se cierre y `event.ignore()` lo cancela:

```
def closeEvent(self, event) -> None:
    self.senal_ventana_cerrada.emit()   # por ejemplo, avisar al back-end
    event.accept()
```

10.1 Mouse

El `event` de los métodos de mouse (`QMouseEvent`):

Método	Retorna
`event.position()`	Posición del clic **relativa al widget**, como `QPointF` (con `.x()` e `.y()` **float**).
`event.position().toPoint()`	La misma posición como `QPoint` (enteros).
`event.globalPosition()`	Posición en la pantalla.
`event.button()`	Qué botón se presionó o soltó: `Qt.MouseButton.LeftButton`, `Qt.MouseButton.RightButton` o `Qt.MouseButton.MiddleButton` (la rueda).
`event.modifiers()`	Qué modificadores estaban apretados (§10.2).

En `wheelEvent`, `event.angleDelta().y()` es positivo al girar hacia adelante y negativo hacia atrás.

```
from PyQt6.QtGui import QMouseEvent

def mousePressEvent(self, event: QMouseEvent) -> None:
    x = event.position().x()
    y = event.position().y()
    if event.button() == Qt.MouseButton.LeftButton:
        ...
```

¿Sobre cuál widget se hizo clic? Un `QLabel` no reacciona a clics, así que el clic sobre un label **llega al `mousePressEvent` de la ventana que lo contiene**. Desde ahí hay tres formas de saber qué label era:

```
def mousePressEvent(self, event: QMouseEvent) -> None:
    # a) Preguntarle a cada label si el cursor está sobre él
    for indice, label in enumerate(self.labels):
        if label.underMouse():
            ...

    # b) Pedirle a la ventana el hijo que está en ese punto
    hijo = self.childAt(event.position().toPoint())    # None si no hay ninguno

    # c) Comparar la posición con la geometría del label
    if self.label.geometry().contains(event.position().toPoint()):
        ...
```

Si necesitas que un clic **no haga nada** en ciertos momentos (por ejemplo, antes de empezar o después de terminar), guarda un atributo `bool` y revísalo al principio del método.

10.2 Teclado

El `event` de `keyPressEvent` y `keyReleaseEvent` (`QKeyEvent`):

Método	Retorna
`event.key()`	Código de la tecla (un `int`), para comparar con las constantes `Qt.Key` de la tabla de abajo.
`event.text()`	El carácter escrito como `str` (`'a'` , `'7'` ...). Es `''` en teclas que no escriben nada, como las flechas o Shift.
`event.modifiers()`	Qué modificadores estaban apretados (tabla de abajo).
`event.isAutoRepeat()`	`True` si el evento se repite por mantener la tecla apretada; `False` en la primera pulsación.

Nombres de las teclas. Todas siguen el patrón `Qt.Key.Key_<Nombre>` :

Teclas	Constantes
Letras	`Qt.Key.Key_A` ... `Qt.Key.Key_Z` (la misma para mayúscula y minúscula), `Qt.Key.Key_Ntilde` (ñ)
Números	`Qt.Key.Key_0` ... `Qt.Key.Key_9`
Flechas	`Qt.Key.Key_Up` , `Qt.Key.Key_Down` , `Qt.Key.Key_Left` , `Qt.Key.Key_Right`
Enter	`Qt.Key.Key_Return` (teclado principal) y `Qt.Key.Key_Enter` (teclado numérico): **son distintas**, revisa las dos
Edición	`Qt.Key.Key_Space` , `Qt.Key.Key_Backspace` , `Qt.Key.Key_Delete` , `Qt.Key.Key_Tab` , `Qt.Key.Key_Escape`
Modificadores	`Qt.Key.Key_Shift` , `Qt.Key.Key_Control` , `Qt.Key.Key_Alt`
Función	`Qt.Key.Key_F1` ... `Qt.Key.Key_F12`
Símbolos	`Qt.Key.Key_Plus` , `Qt.Key.Key_Minus` , `Qt.Key.Key_Asterisk` , `Qt.Key.Key_Slash` , `Qt.Key.Key_Equal` , `Qt.Key.Key_Period` , `Qt.Key.Key_Comma`

Para saber el nombre de una tecla que no está acá, imprímelo al apretarla: `print(Qt.Key(event.key()).name)` muestra, por ejemplo, `Key_A` .

Modificadores (`Qt.KeyboardModifier`): `ShiftModifier` , `ControlModifier` , `AltModifier` . Se revisan con `&` :

```
from PyQt6.QtGui import QKeyEvent

def keyPressEvent(self, event: QKeyEvent) -> None:
    if event.key() == Qt.Key.Key_R:
        self.senal_reiniciar.emit()
    elif event.key() in (Qt.Key.Key_Return, Qt.Key.Key_Enter):
        self.senal_confirmar.emit()
    elif event.key() == Qt.Key.Key_S and event.modifiers() & Qt.KeyboardModifier.ControlModifier:
        self.senal_guardar.emit()      # Ctrl+S
```

Dos detalles:

- Apretar Shift, Ctrl o Alt **también** llama a `keyPressEvent`, con su propio código (`Key_Shift`...). Si solo te interesan letras, compara contra las teclas que esperas en vez de reaccionar a cualquier tecla.
- Las teclas llegan al widget que tiene **el foco**. Si la ventana tiene un `QLineEdit` y el cursor está en él, el campo se queda con las letras y los números (son texto para él), y a la ventana solo le llegan las teclas que el campo no usa, como las flechas arriba y abajo o Esc.

---

11. Separación *front-end / back-end*

Capa	Qué hace	Qué **no** hace
Front-end (`frontend.py`): clases que heredan de `QWidget`	Crea y muestra widgets, captura lo que hace el usuario y lo **emite como señal**; recibe resultados por slots y actualiza la vista.	No valida reglas ni guarda el estado del programa.
Back-end (`backend.py`): clases que heredan de `QObject`	Guarda el estado, valida, aplica las reglas y **emite señales** con el resultado.	No importa ni toca widgets.
`main.py`	Crea la `QApplication`, instancia front y back, **conecta las señales** de uno con los métodos del otro y muestra la primera ventana.	No tiene lógica.

El objetivo es **alta cohesión** (cada clase hace una sola cosa) y **bajo acoplamiento** (el front no conoce al back ni al revés; solo se hablan por señales). Así se puede cambiar la interfaz sin tocar la lógica.

Flujo típico de una acción:

```
usuario hace clic ──► Frontend: slot del clic
                        └─ emite senal_pedir(datos)
                                  │   (conectada en main.py)
                                  ▼
                      Backend: valida y actualiza su estado
                        ├─ si hay error: emite senal_error(mensaje)
                        └─ si no:        emite senal_actualizar(datos)
                                  │   (conectadas en main.py)
                                  ▼
                      Frontend: slot que actualiza la vista
```

Esqueleto de los tres archivos:

```python
# backend.py
from PyQt6.QtCore import QObject, pyqtSignal


class Logica(QObject):
    senal_actualizar = pyqtSignal(int)
    senal_error = pyqtSignal(str)

    def __init__(self) -> None:
        super().__init__()
        self.contador = 0

    def sumar(self, cantidad: int) -> None:
        if cantidad <= 0:
            self.senal_error.emit('La cantidad debe ser positiva')
            return
        self.contador += cantidad
        self.senal_actualizar.emit(self.contador)
```

```python
# frontend.py
from PyQt6.QtWidgets import QWidget, QLabel, QLineEdit, QPushButton, QVBoxLayout
from PyQt6.QtCore import pyqtSignal


class Ventana(QWidget):
    senal_sumar = pyqtSignal(int)

    def __init__(self) -> None:
        super().__init__()
        self.setWindowTitle('Contador')
        self.label = QLabel('0', self)
        self.campo = QLineEdit('', self)
        self.boton = QPushButton('&Sumar', self)
        self.boton.clicked.connect(self.enviar)
        layout = QVBoxLayout()
        layout.addWidget(self.label)
        layout.addWidget(self.campo)
        layout.addWidget(self.boton)
        self.setLayout(layout)

    def enviar(self) -> None:
        texto = self.campo.text()
        if texto.isdigit():
            self.senal_sumar.emit(int(texto))
        else:
            self.label.setText('Escribe un número')

    def actualizar(self, valor: int) -> None:
        self.label.setText(str(valor))

    def mostrar_error(self, mensaje: str) -> None:
        self.label.setText(mensaje)
```

```
# main.py
import sys
from PyQt6.QtWidgets import QApplication

from frontend import Ventana
from backend import Logica


if __name__ == '__main__':
    def hook(type_, value, traceback) -> None:
        sys.__excepthook__(type_, value, traceback)

    sys.excepthook = hook

    app = QApplication([])

    ventana = Ventana()
    logica = Logica()

    ventana.senal_sumar.connect(logica.sumar)
    logica.senal_actualizar.connect(ventana.actualizar)
    logica.senal_error.connect(ventana.mostrar_error)

    ventana.show()
    sys.exit(app.exec())
```

> Una convención que ayuda a leer `main.py`: nombrar las señales del back-end según la ventana a la que van (`senal_v1_...`, `senal_v2_...`).

---

12. Varias ventanas

Cada ventana es una clase que hereda de `QWidget`. La forma recomendada:

1. **Instanciar todas las ventanas en `main.py`**, antes de `app.exec()`, y guardarlas en variables: una ventana sin ninguna referencia viva desaparece al instante.
2. **Mostrar solo la primera** con `show()`.
3. **Cambiar de ventana con señales**: la ventana que se va se oculta con `hide()` y emite una señal conectada al `show` (o a un método que también recibe datos) de la siguiente.

```
# frontend.py
class VentanaInicio(QWidget):
    senal_continuar = pyqtSignal(str)

    def continuar(self) -> None:
        self.hide()
        self.senal_continuar.emit(self.campo_nombre.text())


class VentanaJuego(QWidget):
    def empezar(self, nombre: str) -> None:
        self.label_nombre.setText(nombre)
        self.show()
```

```
# main.py
ventana_inicio = VentanaInicio()
ventana_juego = VentanaJuego()

ventana_inicio.senal_continuar.connect(ventana_juego.empezar)

ventana_inicio.show()
sys.exit(app.exec())
```

Si es el **back-end** quien decide si se puede pasar a la siguiente ventana (porque hay que validar algo), la ventana emite su señal hacia el back-end, y el back-end emite una señal conectada a `ventana_inicio.hide` y otra conectada a `ventana_juego.empezar`; si no se puede, emite una de error hacia la ventana de inicio.

Lo que **no** conviene: que una ventana cree a la otra dentro de un método (`self.otra = OtraVentana()`). Acopla las dos clases y, si la referencia es local, la ventana nueva se cierra sola.

---

13. PyQt5 vs PyQt6

Lo que vas a encontrar distinto en ejemplos de internet:

PyQt5	PyQt6
`from PyQt5.QtWidgets import ...`	`from PyQt6.QtWidgets import ...`
`app.exec_()`	`app.exec()`
`Qt.AlignCenter`	`Qt.AlignmentFlag.AlignCenter`
`Qt.KeepAspectRatio`	`Qt.AspectRatioMode.KeepAspectRatio`
`Qt.Key_A`	`Qt.Key.Key_A`
`Qt.LeftButton`	`Qt.MouseButton.LeftButton`
`QLineEdit.Password`	`QLineEdit.EchoMode.Password`
`event.pos()`, `event.x()`	`event.position()`, `event.position().x()`
`QAction` en `QtWidgets`	`QAction` en `QtGui`

Regla general: en PyQt6 las constantes van con el **nombre de su enum** en medio (`Qt.<Enum>.<Valor>`).

---

14. Otros widgets útiles

Todos están en `PyQt6.QtWidgets` .

Widget	Para qué	Métodos y señales más usados
`QTextEdit`	Texto de varias líneas	`toPlainText()` , `setPlainText(t)` , `append(t)` , `setReadOnly(True)`
`QCheckBox`	Casilla marcable	`isChecked()` , `setChecked(b)` , señal `stateChanged`
`QRadioButton`	Una opción entre varias (las del mismo padre se excluyen)	`isChecked()` , señal `toggled(bool)`
`QComboBox`	Lista desplegable	`addItem(t)` , `addItems(lista)` , `currentText()` , `currentIndex()` , señal `currentIndexChanged(int)`
`QListWidget`	Lista de elementos	`addItem(t)` , `currentItem().text()` , `clear()` , señal `itemClicked`
`QSpinBox`	Entero con flechas	`setRange(a, b)` , `value()` , `setValue(n)` , señal `valueChanged(int)`
`QSlider`	Deslizador	`setRange(a, b)` , `value()` , señal `valueChanged(int)`
`QProgressBar`	Barra de progreso	`setRange(a, b)` , `setValue(n)`
`QMessageBox`	Ventana emergente de aviso	`QMessageBox.information(self, 'Título', 'Texto')` , también `warning` , `critical` y `question` (retorna el botón apretado, p. ej. `QMessageBox.StandardButton.Yes`)

---

15. Buscar lo que no está acá

PyQt6 trae su propia información, así que se puede explorar sin internet desde un intérprete de Python (`python` en la terminal) o con un `print` dentro del programa:

Quiero saber…	Escribe	Resultado (ejemplo)
Cómo se llama un método y qué recibe	`print(QLabel.setPixmap.__doc__)`	`setPixmap(self, a0: QPixmap)`
Qué métodos tiene una clase	`[m for m in dir(QLabel) if 'pixmap' in m.lower()]`	`['pixmap', 'setPixmap']`
Todos los `set…` de una clase	`[m for m in dir(QLabel) if m.startswith('set')]`	`['setAlignment', …]`
Qué señales tiene un widget	`[n for n in dir(QLineEdit) if type(getattr(QLineEdit, n)).__name__ == 'pyqtSignal']`	`[…, 'returnPressed', 'textChanged', …]`
Qué valores tiene una constante	`[x.name for x in QLineEdit.EchoMode]`	`['Normal', 'NoEcho', 'Password', 'PasswordEchoOnEdit']`
Qué tecla se apretó	`print(Qt.Key(event.key()).name)` dentro de `keyPressEvent`	`Key_Return`

Quiero saber…	Escribe	Resultado (ejemplo)
Qué eventos puede sobrescribir un widget	`[m for m in dir(QWidget) if m.endswith('Event')]`	`[…, 'keyPressEvent', 'mousePressEvent', …]`

Antes, los imports de la clase que quieras revisar (`from PyQt6.QtWidgets import QLabel`).

> Ojo: el `__doc__` escribe las constantes **con el estilo de PyQt5**. Por ejemplo, `print(QPixmap.scaled.__doc__)` muestra `aspectRatioMode: Qt.AspectRatioMode = Qt.IgnoreAspectRatio`. En tu código se usa el nombre completo: `Qt.AspectRatioMode.IgnoreAspectRatio` (§13). El tipo antes del `=` (`Qt.AspectRatioMode`) te dice dónde buscar.